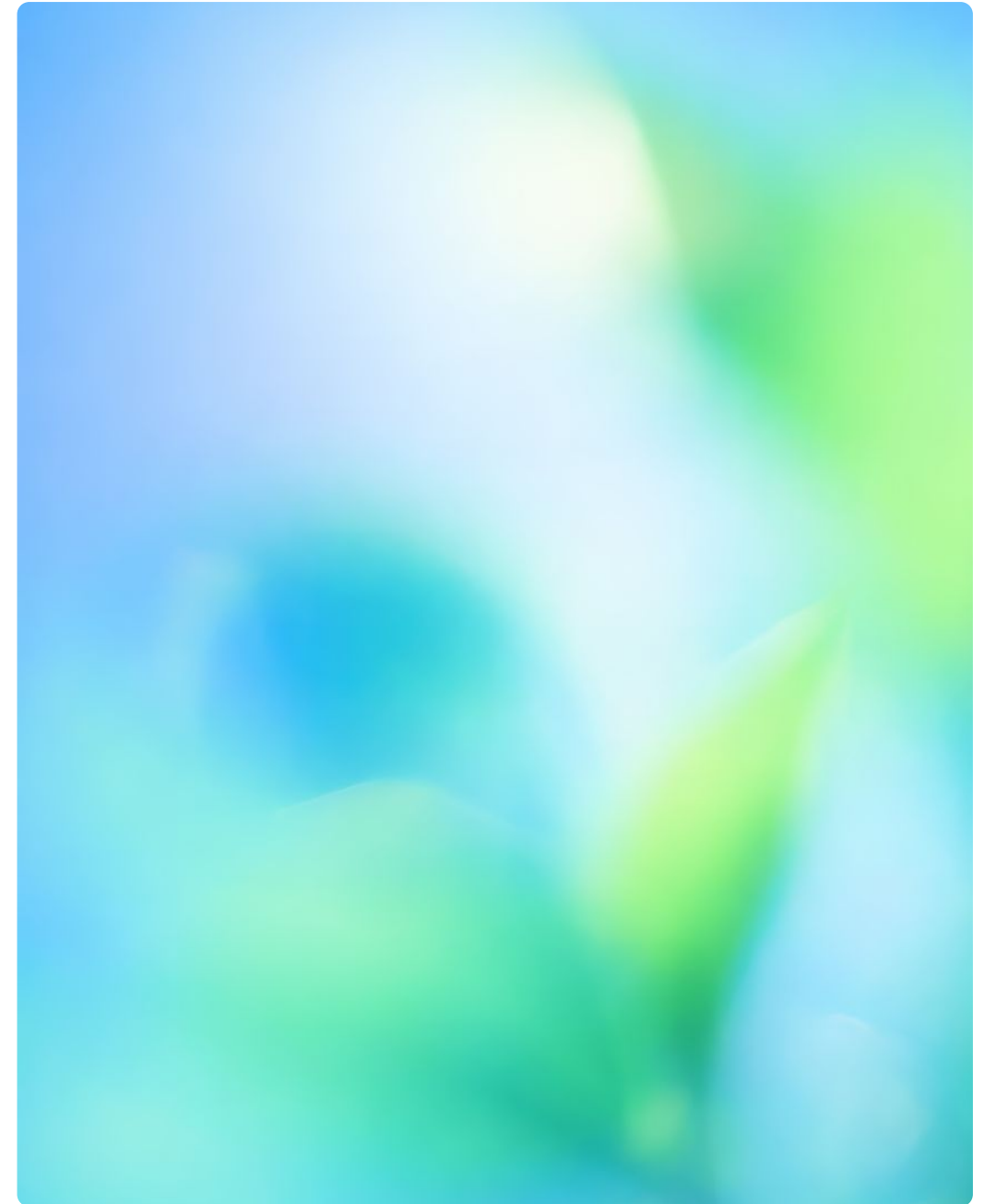


# You Can Just Build ~~Things~~ ... Chips

Richard Ho, VP Hardware

Ravi Narayanaswami, Member of Technical Staff

Chris Leary, Member of Technical Staff



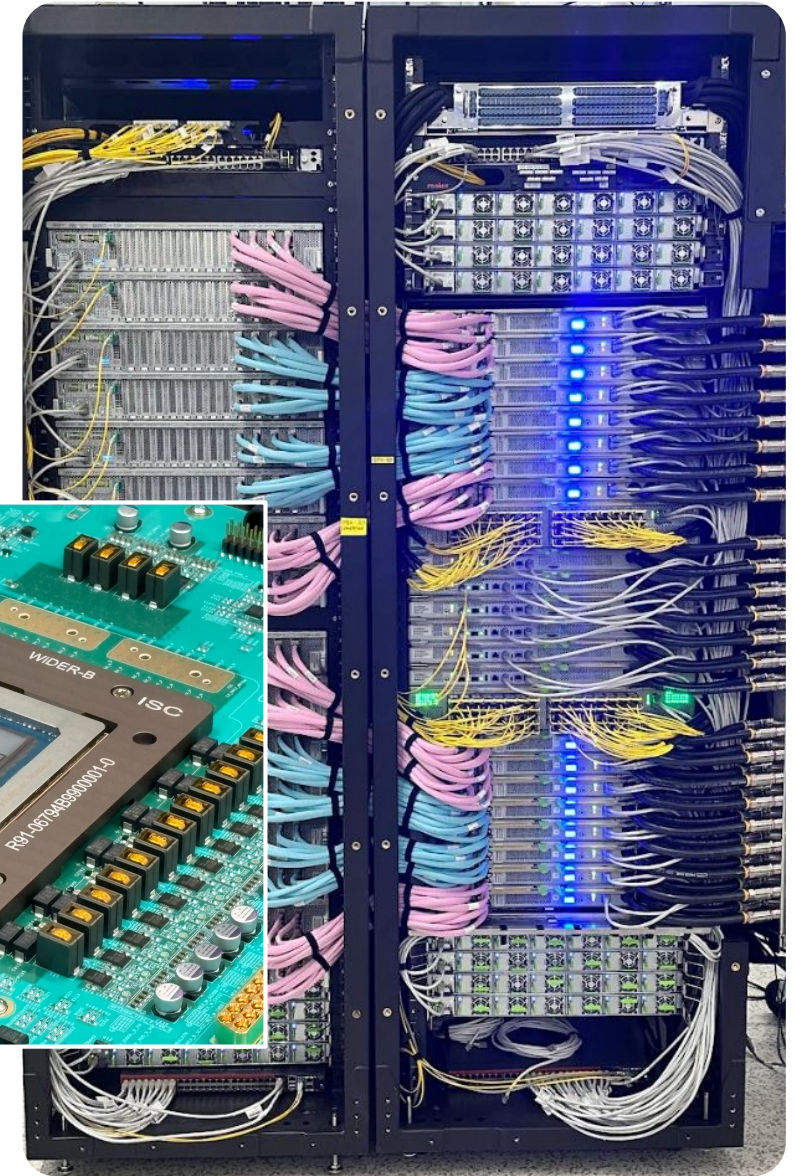
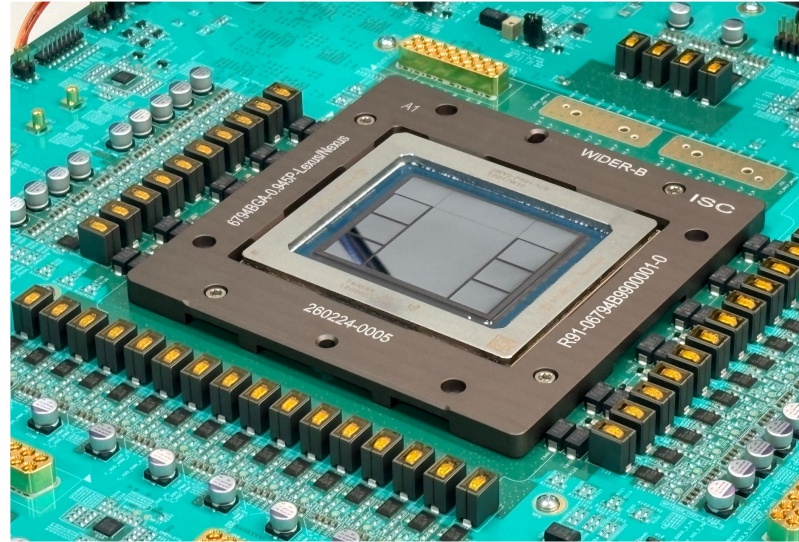
# Jalapeño ASIC + System

Designed to be the best compute platform for OpenAI's inference workloads

From initial RTL to tapeout in 9 months

AI accelerated HW+SW co-design

Early silicon + system:  
SoTA perf/W @ low latency for multi-chip workloads

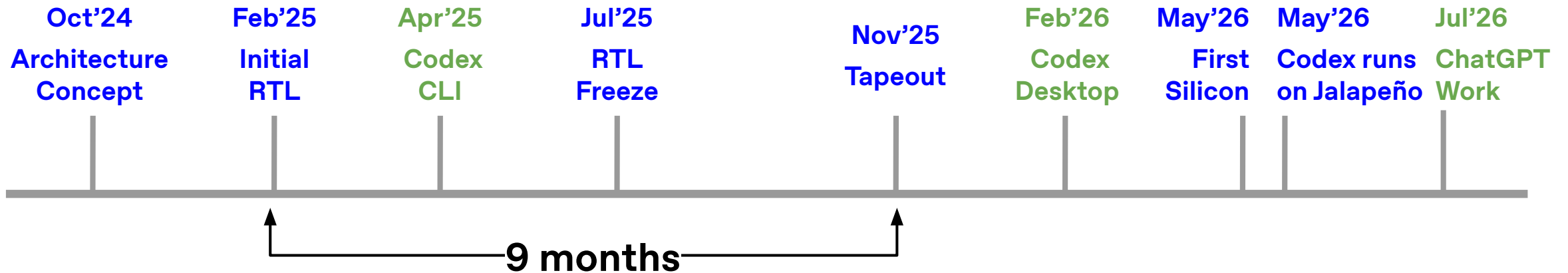


Photos of Jalapeño ASIC and host + accelerator rack pair

# Project timeline

Starting point: How to optimize compute for OpenAI's workloads?

Once the importance of inference and agentic workloads came into focus, the need for a new design was clear



# Performance

OpenAI



# Metrics that matter: latency and energy

## 01 · USER EXPERIENCE

### Request latency

Time to complete an inference request

**TTLT · time to last token**

End-to-end latency; TBT is another useful proxy

## 02 · INFERENCE EFFICIENCY

### Energy per request

Energy needed to process an inference request

**Energy / token · tokens / joule**

Also measured as tokens/s/kW

### Single user latency ↔ energy per token

Lower latency usually costs more energy per token.

Energy is a proxy for cost

Compare systems across the full Pareto frontier.

EFFICIENCY · TOKENS/JOULE ↑



REQUEST LATENCY · TTLT →

NON GOALS

~~# chips used~~

~~throughput per chip~~

~~TTF~~

# Compare efficiency @ matched user experience



# SemiAnalysis InferenceX: public, power-normalized comparisons

## Representative OSS model basket

Compares across multiple OSS models—not one architecture or a handpicked kernel.

## Open to hardware-specific optimization

Available across platforms, so each vendor can tune its full software and hardware stack.

## Covers full pareto of prefill, decode

All phases of inference workloads are covered  
Covers the full latency-throughput spectrum

## Measures the end-to-end request path

Captures request-level execution, rather than only peak FLOPS or isolated kernel performance.

## Compare Jalapeño to July 2026 Pareto frontier across accelerators

InferenceX · nominal ISL/OSL 8k/1k · weight dtype f4 · package-TDP normalized  
Jalapeño = 700 W, GB200 = 1.2 kW, GB300 = 1.4 kW, MI355X = 1.4 kW  
Vendor software stack: see the published InferenceX run configuration

# Speculative decoding: 8 tokens, 1 large-model pass

STP = single-token prediction

MTP = multi-token prediction

## STP · single-token prediction

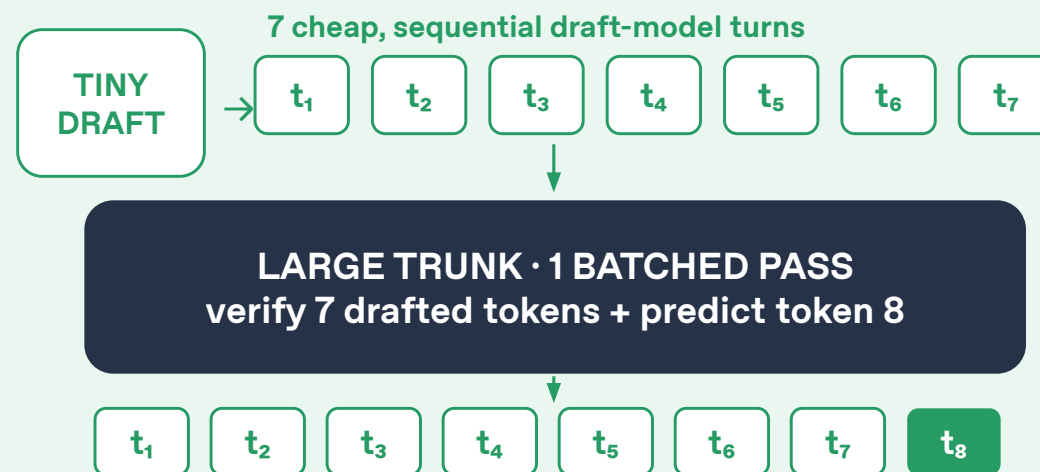
8 serial forward passes through the large trunk



8 expensive weight reads · 8 serial trunk steps

## MTP = 7 · multi-token prediction

7 tiny draft turns + 1 batched large-trunk pass



1 expensive trunk pass · up to 8 output tokens

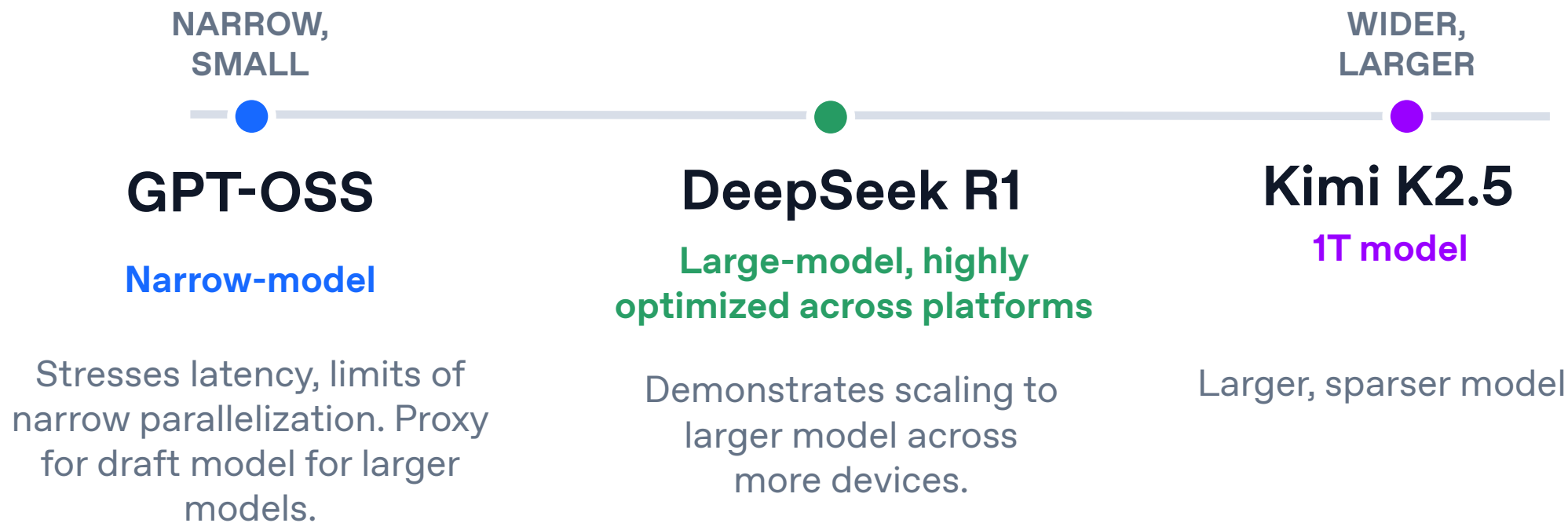
**Up to 8× fewer large-model passes → lower latency + higher throughput**

MTP = 7: verify seven drafted tokens + predict one bonus token in a single batched trunk pass.

Benchmark context: Jalapeño uses STP; selected comparison baselines use MTP.

Actual speedup depends on acceptance rate, draft overhead, and verification efficiency.

# Three models to demonstrate generality and velocity



- Models not co-designed for Jalapeño
- Jalapeño results use STP; selected baselines include MTP
- Nominal ISL/OSL = 8k/1k; weight dtype = f4

**Demonstrated velocity:** We got this all running between when A0 came back to the lab and now

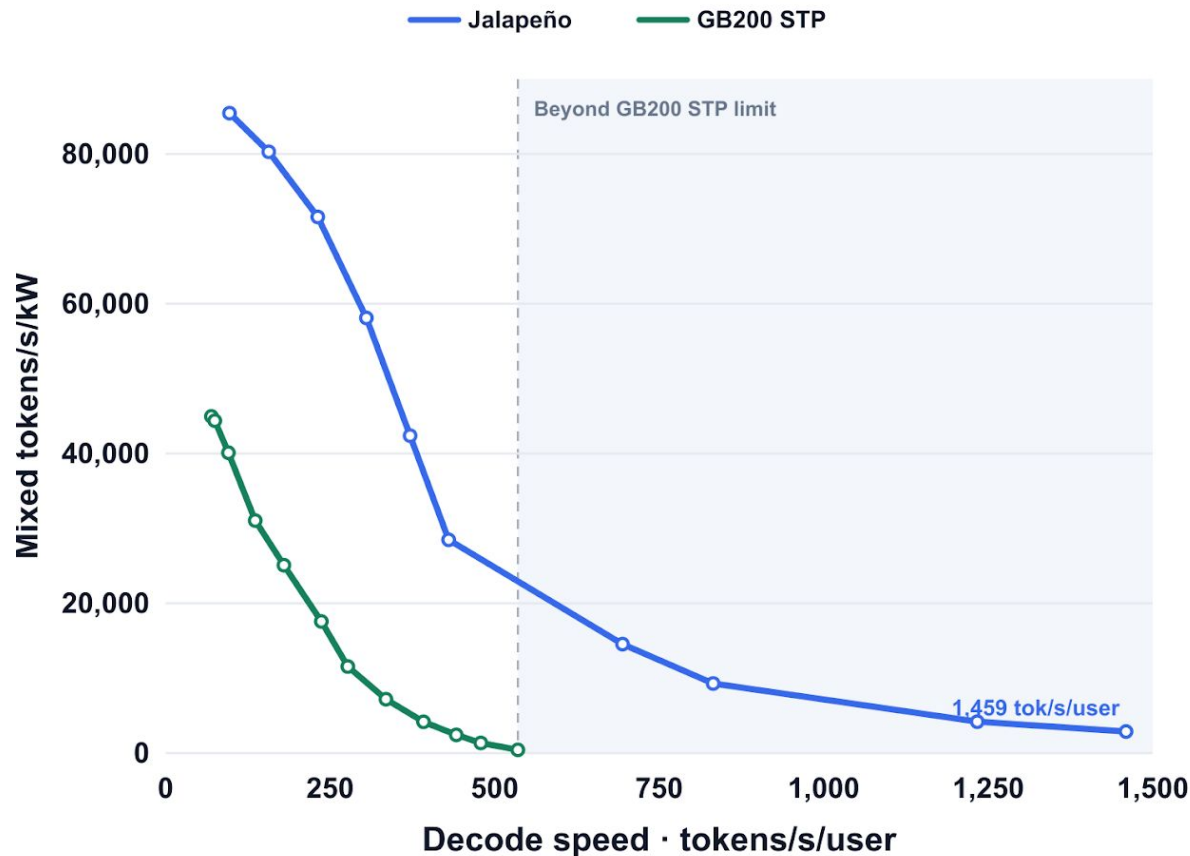


# Jalapeño is pareto frontier at GPT-OSS 120B

InferenceX · GPT-OSS-120B · nominal 8k/1k · STP · package TDP: Jalapeño 700 W; GB200 1,200 W

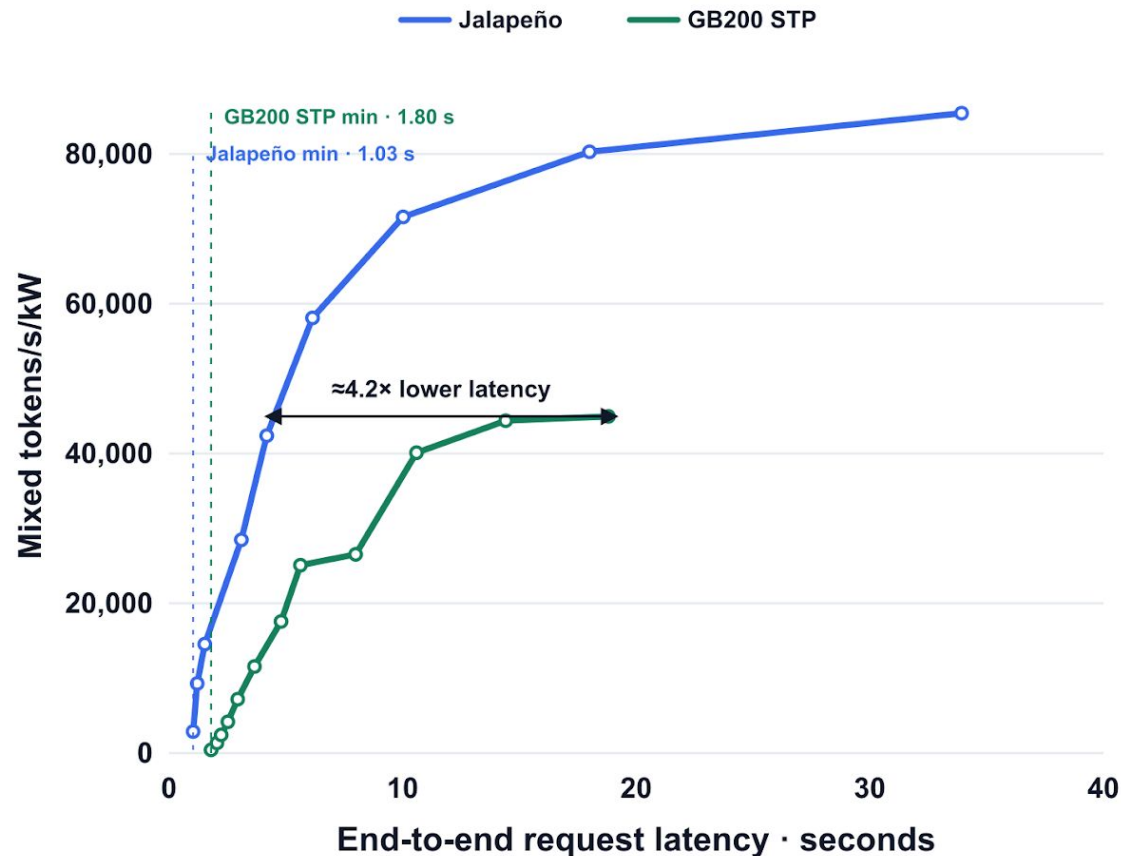
## MIXED THROUGHPUT VS. INTERACTIVITY

Efficiency across decoding speeds



## MIXED THROUGHPUT VS. END-TO-END LATENCY

Efficiency across full-request latency

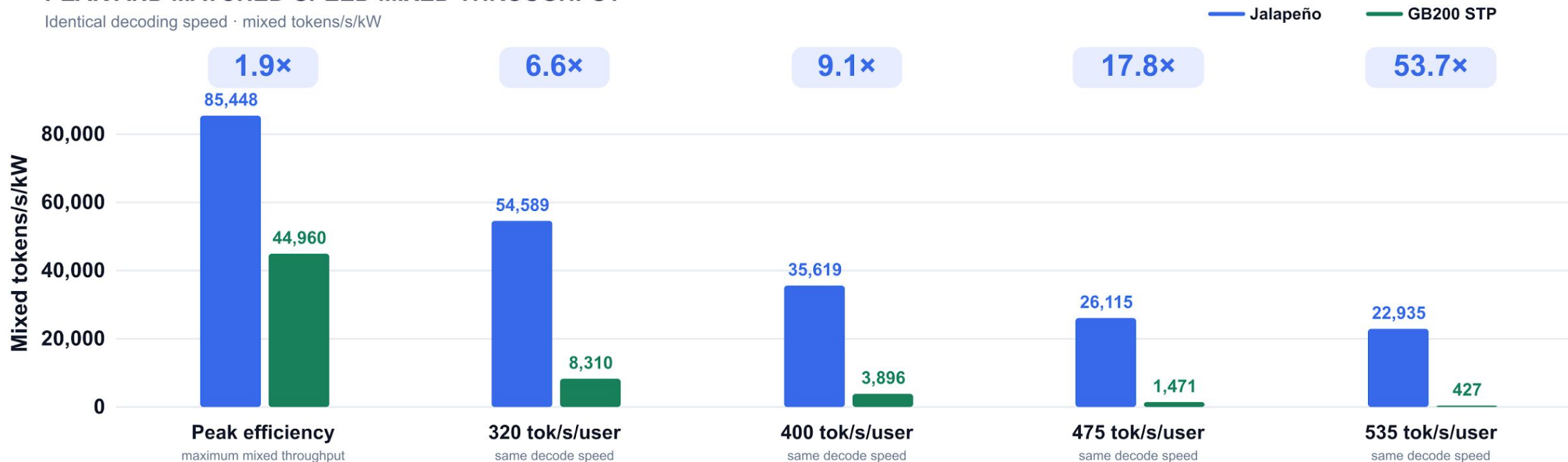


# Jalapeño leads across GPT-OSS operating points

InferenceX · GPT-OSS-120B · nominal 8k/1k · STP · package TDP: Jalapeño 700 W; GB200 1,200 W

## PEAK AND MATCHED-SPEED MIXED THROUGHPUT

Identical decoding speed · mixed tokens/s/kW



≈1.9x

higher peak mixed TPS / kW

85,448 vs. 44,960 mixed / kW

≈1.7x

lower end-to-end latency

1.03 s vs. 1.80 s

≈2.7x

lower min TBT

0.69 vs. 1.87 ms  
(1,459 vs. 535 tok/s/user)

≈53.7x

more throughput at previous TBT

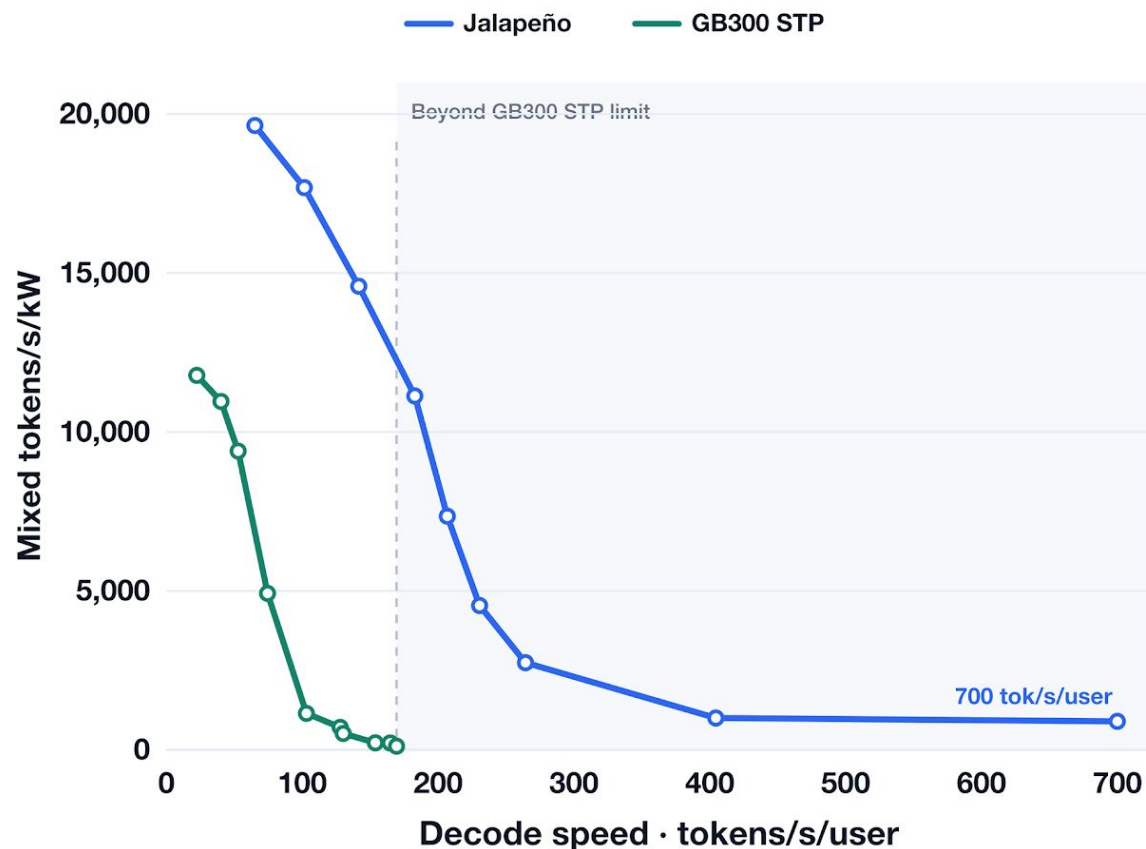
22,935 vs. 427 mixed / kW  
(at 535.28 tok/s/user)

# Jalapeño is pareto frontier at DeepSeek R1 670B

InferenceX · DeepSeek R1 MXFP4 · nominal 8k/1k · STP · package TDP: Jalapeño 700 W; GB300 1,400 W

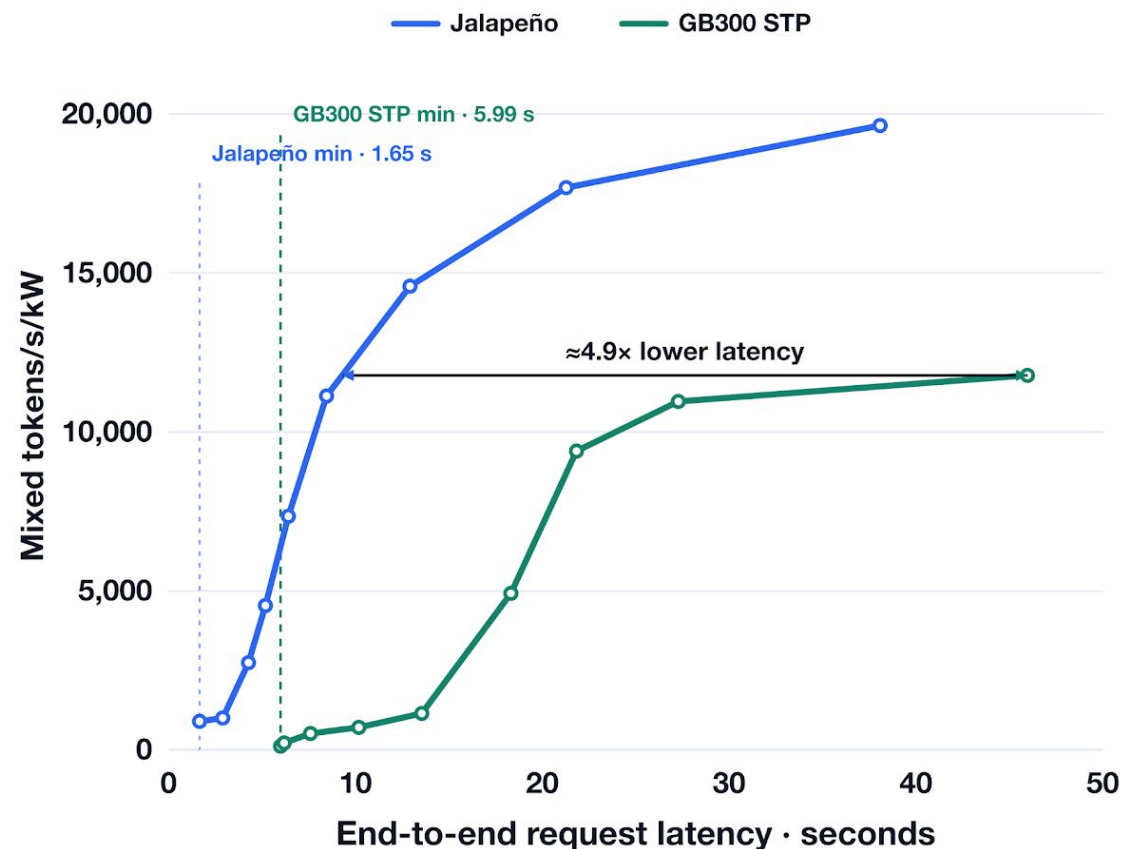
## MIXED THROUGHPUT VS. INTERACTIVITY

Efficiency across decoding speeds



## MIXED THROUGHPUT VS. END-TO-END LATENCY

Efficiency across full-request latency

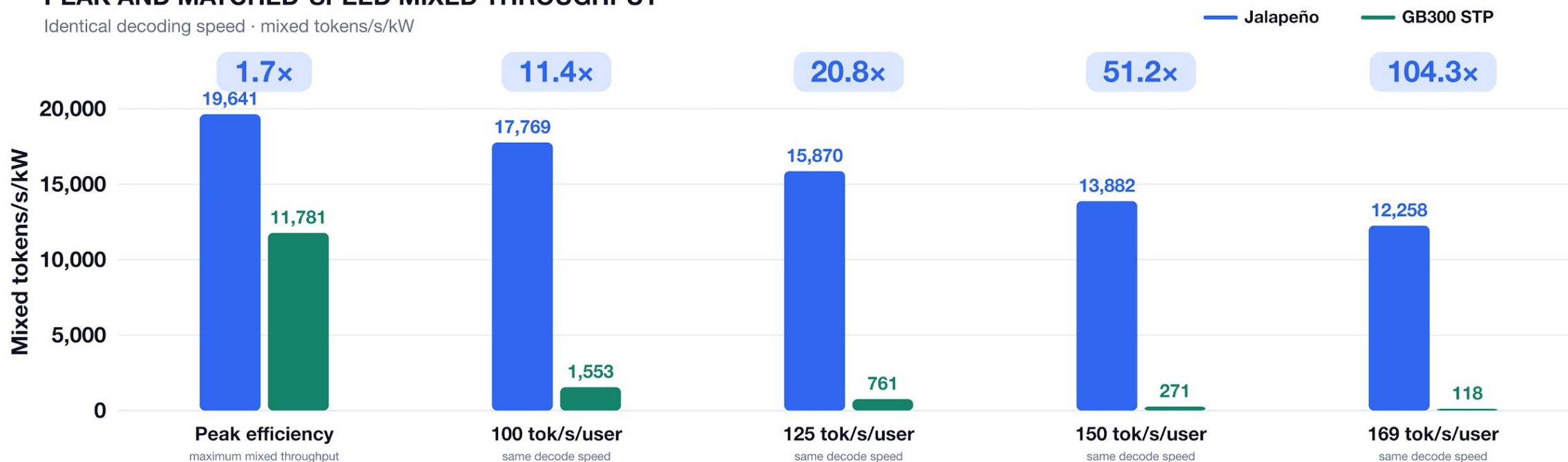


# Jalapeño leads across DeepSeek R1 operating points

InferenceX · DeepSeek R1 MXFP4 · nominal 8k/1k · STP · package TDP: Jalapeño 700 W; GB300 1,400 W

## PEAK AND MATCHED-SPEED MIXED THROUGHPUT

Identical decoding speed · mixed tokens/s/kW



≈1.7x

higher peak mixed TPS / kW

19,641 vs. 11,781 mixed / kW

≈3.6x

lower end-to-end latency

1.65 s vs. 5.99 s

≈4.1x

lower min TBT

1.43 vs. 5.90 ms  
(700 vs. 169 tok/s/user)

≈104.3x

more throughput at previous TBT

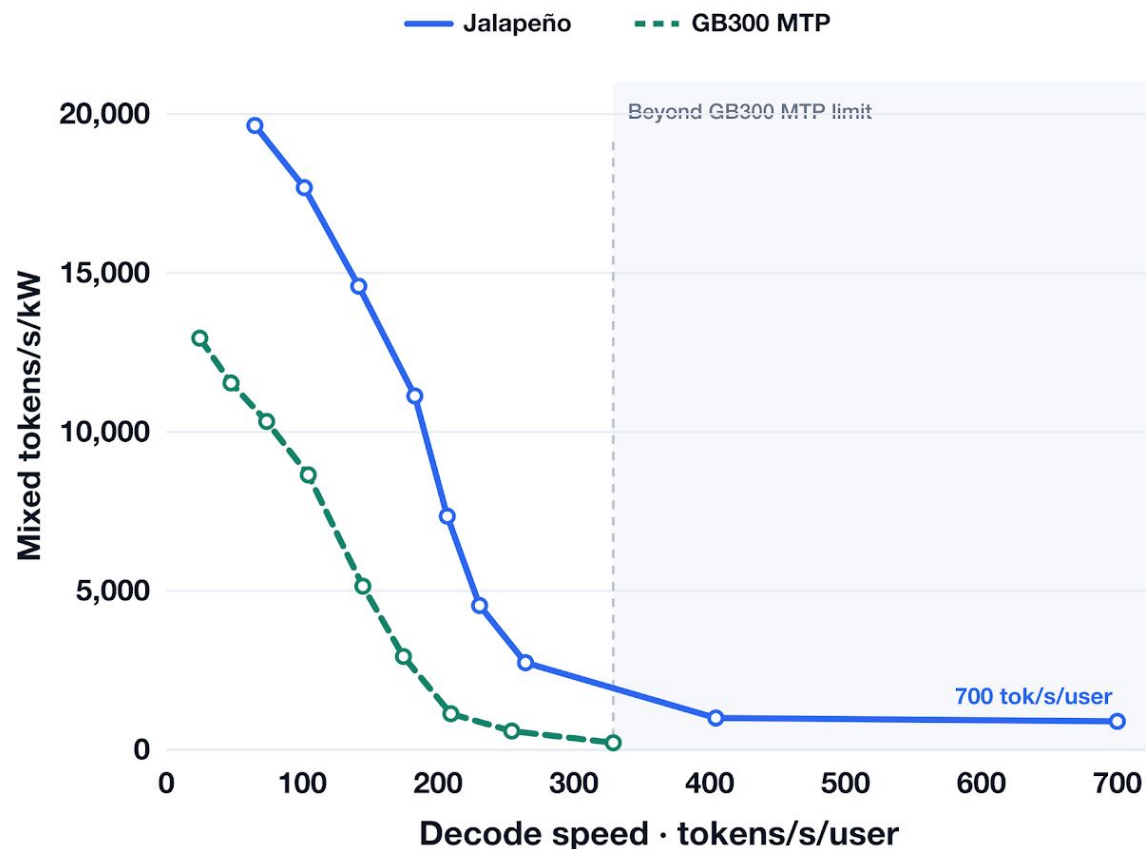
12,258 vs. 118 mixed / kW  
(at 169.41 tok/s/user)

# Jalapeño STP advances the frontier vs. GB300 MTP

InferenceX · DeepSeek R1 MXFP4 · nominal 8k/1k · Jalapeño STP vs. GB300 MTP · TDP: 700 W / 1,400 W

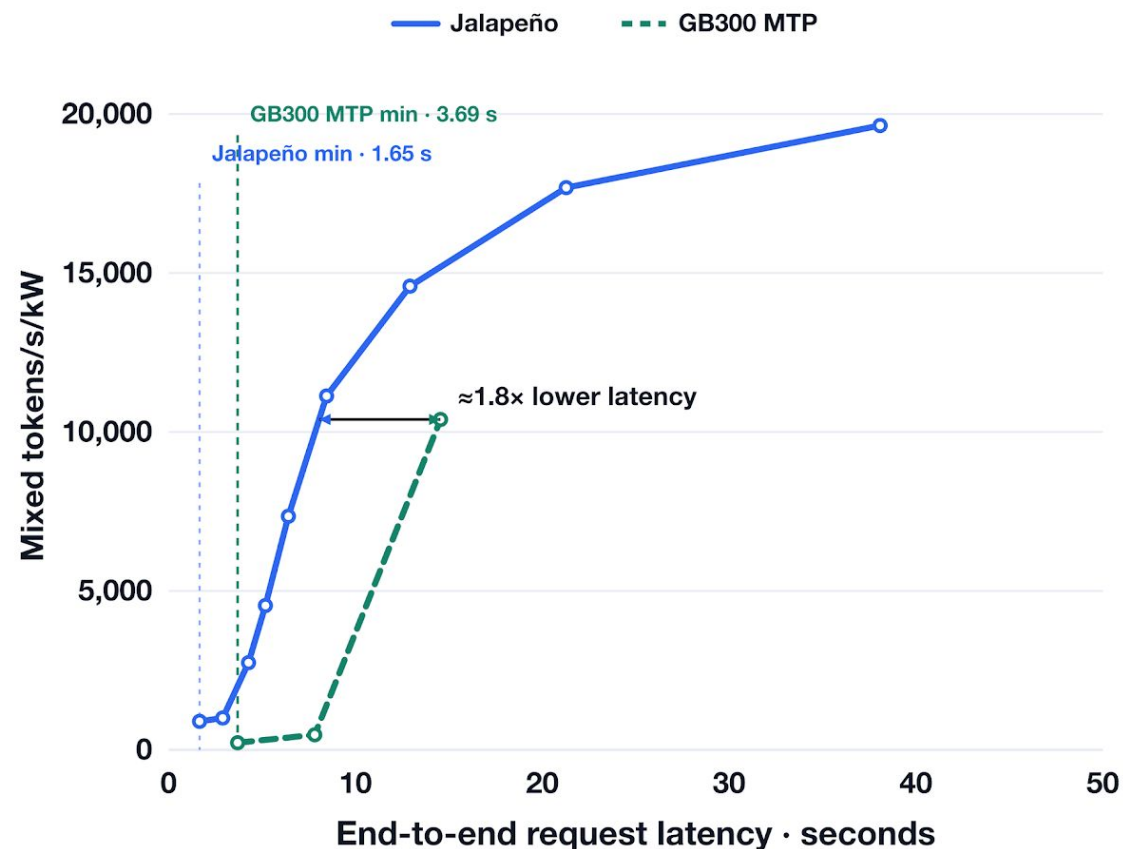
## MIXED THROUGHPUT VS. INTERACTIVITY

Efficiency across decoding speeds



## MIXED THROUGHPUT VS. END-TO-END LATENCY

Efficiency across full-request latency



# Jalapeño STP leads across GB300 MTP operating points

InferenceX · DeepSeek R1 MXFP4 · nominal 8k/1k · Jalapeño STP vs. GB300 MTP · TDP: 700 W / 1,400 W

## PEAK AND MATCHED-SPEED MIXED THROUGHPUT

Identical decoding speed · mixed tokens/s/kW



≈1.5x

higher peak mixed TPS / kW

19,641 vs. 12,951 mixed / kW

≈2.2x

lower end-to-end latency

1.65 s vs. 3.69 s

≈2.1x

lower min TBT

1.43 vs. 3.04 ms  
(700 vs. 329 tok/s/user)

≈8.6x

more throughput at previous TBT

1,940 vs. 225 mixed / kW  
(at 328.95 tok/s/user)

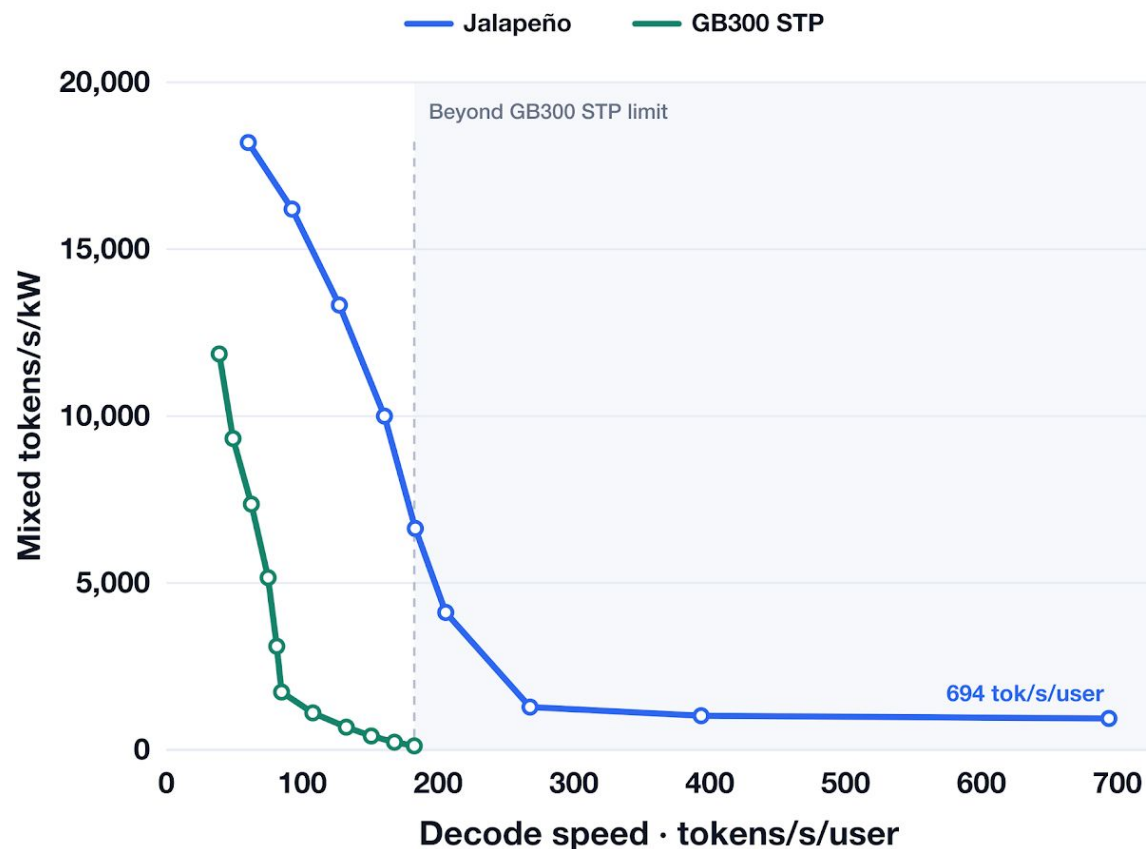


# Jalapeño is pareto frontier at Kimi K2.5 1T

InferenceX · Kimi K2.5 MXFP4 · nominal 8k/1k · STP · package TDP: Jalapeño 700 W; GB300 1,400 W

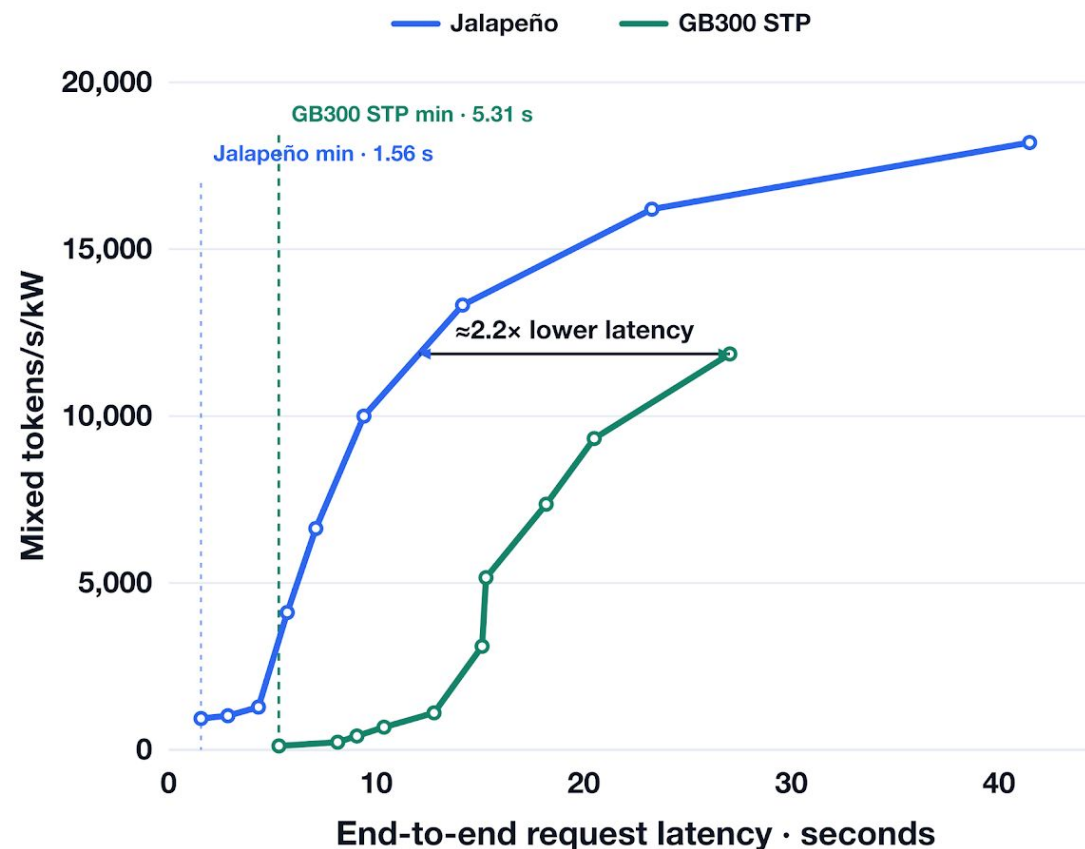
## MIXED THROUGHPUT VS. INTERACTIVITY

Efficiency across decoding speeds



## MIXED THROUGHPUT VS. END-TO-END LATENCY

Efficiency across full-request latency



# Jalapeño leads across Kimi K2.5 operating points

InferenceX · Kimi K2.5 MXFP4 · nominal 8k/1k · STP · package TDP: Jalapeño 700 W; GB300 1,400 W

## PEAK AND MATCHED-SPEED MIXED THROUGHPUT

Identical decoding speed · mixed tokens/s/kW



≈1.5x

higher peak mixed TPS / kW

18,195 vs. 11,862 mixed / kW

≈3.4x

lower end-to-end latency

1.56 s vs. 5.31 s

≈3.8x

lower min TBT

1.44 vs. 5.48 ms  
(694 vs. 182 tok/s/user)

≈56.1x

more throughput at previous TBT

6,744 vs. 120 mixed / kW  
(at 182.46 tok/s/user)

# Key Takeaways

| Performance Metrics         |               |        |
|-----------------------------|---------------|--------|
| Interactivity               | 2.1x – 4.1x   | Higher |
| E2E latency                 | 1.7x – 3.6x   | Lower  |
| Perf/W at previous-best TBT | 8.6x – 104.3x | Higher |
| Perf/W at peak throughput   | 1.5x – 1.9x   | Higher |

## Scaling for OpenAI models

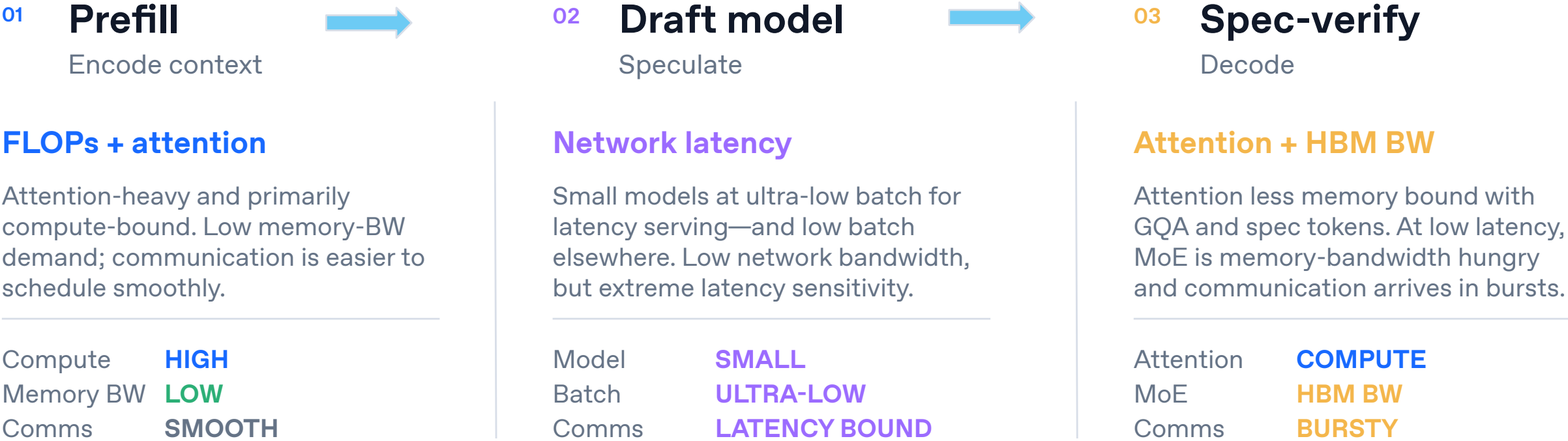
- Advantage widens for internal models
- Multi-token prediction boosts latency another 3-5x (iso-efficiency)
- < 1ms TBT on frontier models at economical throughputs

# Agentic workloads

OpenAI



# A request spans three distinct hardware regimes



## What matters is requests/second/watt at the required SLA latency

Each phase hits a different bottleneck; efficiency only counts if the complete request remains within its end-to-end latency target.

# Changing ratios → inefficiency in heterogeneous fleets

Example of changing workload ratios

● Prefill    ● Draft    ● Verify



## Ratio affected by

- Model architecture
- Attention algorithms
- Token efficiency
- Speculative acceptance
- Context length
- Latency vs. throughput

## Specialized, heterogeneous fleet

Underused chips still draw power, rack space, incurring high baseline costs

Prefill (Active)

Draft (Idle)

Verify (Idle)

## Single balanced chip

Silicon utilization varies by phase, unused blocks are gated

SINGLE BALANCED CHIP

Compute

Memory (Gated)

Network (Gated)

## Observation: dark silicon is cheaper than idle accelerators

Separate accelerators still pay baseline package, HBM, I/O, network, and cooling power; inactive blocks on one chip can be gated while KV stays local.



# System choice: KV moves or the active silicon mix varies

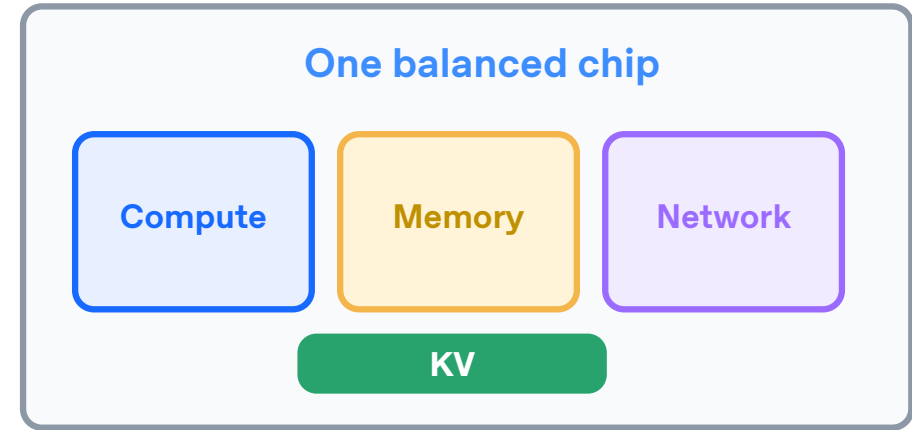
## Disaggregated specialization



Network + synchronization at every phase boundary

Specialized hardware, but KV state must move between systems

## Unified, internally heterogeneous



Activate the right ratio by phase; unused units go dark

Heterogeneity moves inside the chip—not across a KV-moving network.

**Locality is king: keep KV local and activate the right resources**

The chip changes the active mix of compute, memory, and network for each phase while unused resources go dark.

# Jalapeño Architecture

OpenAI



# Raw HBM bandwidth implies extraordinary token rates

$$\begin{array}{ccccc} \mathbf{1+ PB/s} & \div & \mathbf{0.5 TB} & = & \mathbf{2,000+} \\ \text{128-chip aggregate} & & \text{1T parameters} \times 4 \text{ bits} & & \text{full-model weight reads / second} \\ \text{HBM4 bandwidth} & & \text{per full model read} & & \end{array}$$

---

**THEORETICAL BANDWIDTH-ONLY CEILING · NOT ACHIEVED**

**1000 - 2,000**

**tokens / s / user**

without speculative decoding

**5,000 - 10,000**

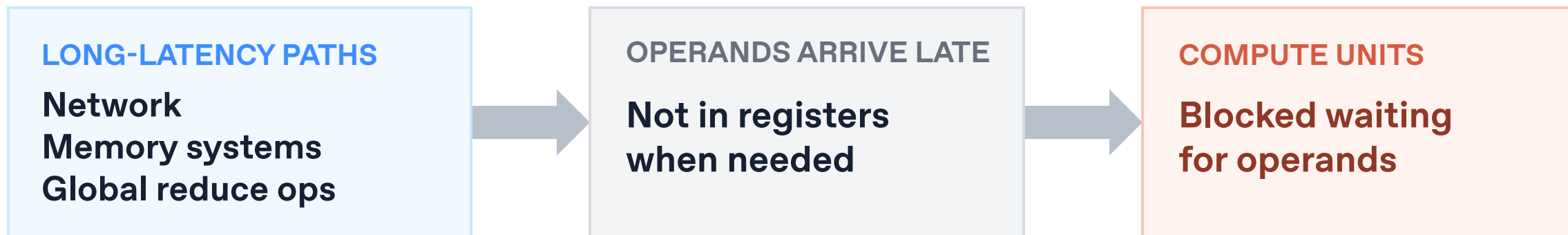
**tokens / s / user**

with speculative decoding

**Reality: we are nowhere close to either rate**

The gap tells us that raw HBM bandwidth is not the sole limiting resource

# Architecture can dominate observed latency



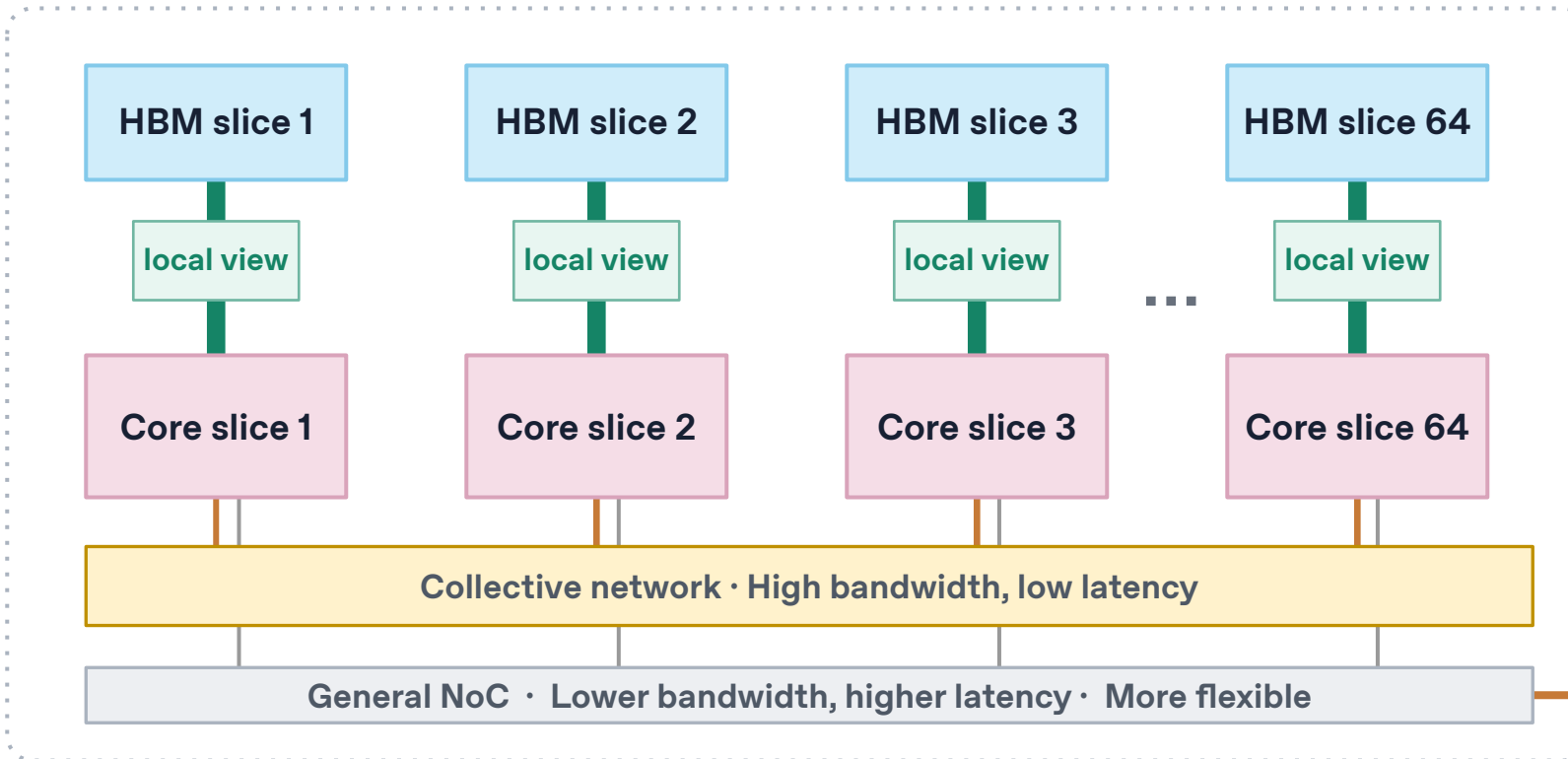
## What causes long latencies and late operands?

Unified memory subsystems tend to highly contended paths  
Independent cores out of sync make global memory fences expensive  
Centralized resources mediating network access

→ **NOT hardware speed of light**

**Utilization falls when compute/memory blocked waiting on data in transit**

# Co-design the architecture to address the bottleneck



## FASTEST LOCAL PATH

Pair each core slice with an HBM slice to expose a low-latency, high-bandwidth local view.

## SPECIALIZED CROSS-CORE

Collective network optimizes bandwidth and latency of common comms patterns.

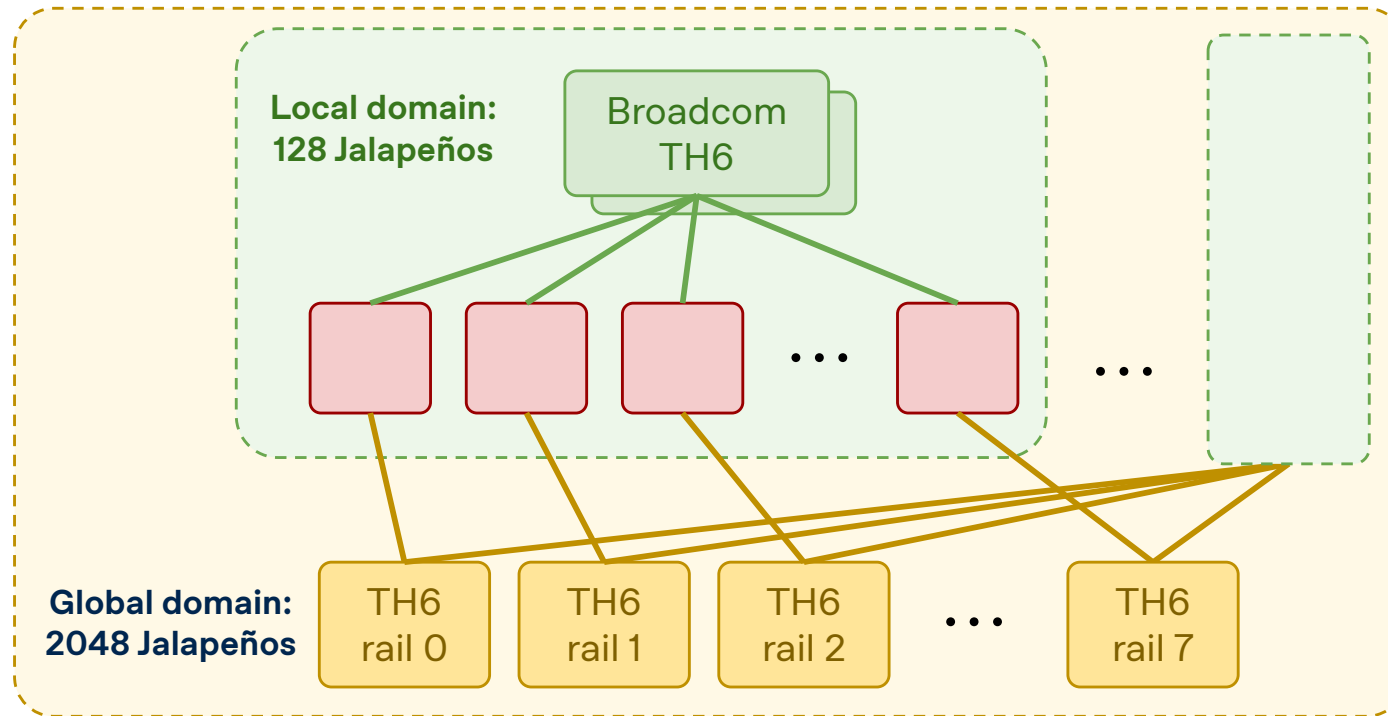
## PRESERVE FLEXIBILITY

Shared NoC for general comms and access to scale-up network.

**Keep common operands local; use the shared path only when necessary**

Explicit placement and distributed control prevent data movement/synchronization from dominating execution

# A single chip isn't enough—the network makes it a system



“Half flattened” two-level Clos topology

## Large scale-up domain

**Fit the entire problem.** Split local and global domains, then taper bandwidth to optimize power.

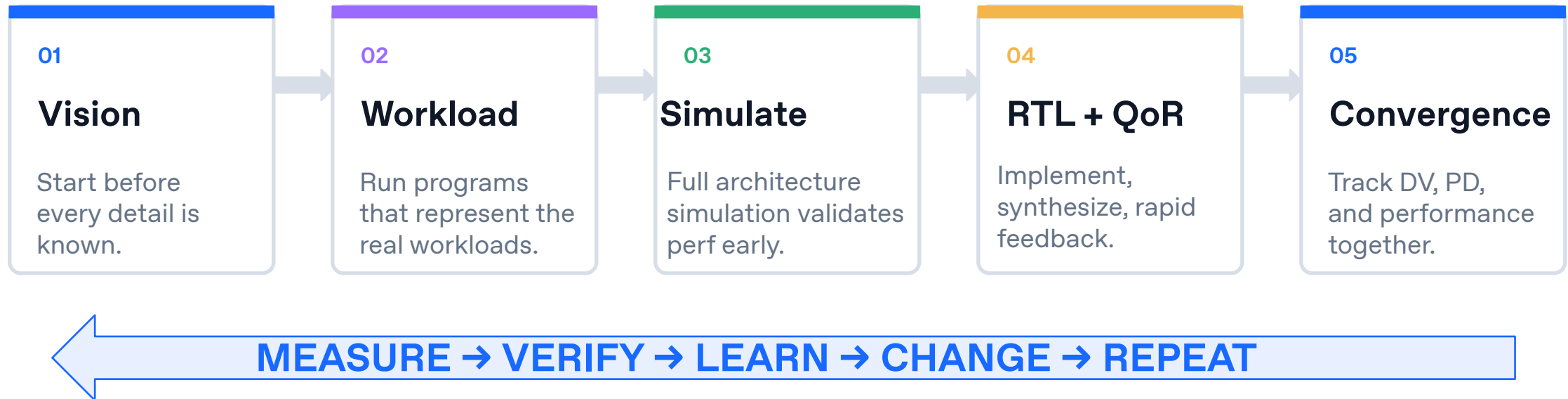
## Low core-to-core latency

Measured from result available at producing core to result uses at consuming core.

**Higher BW for Tensor Parallel. Lower BW for Expert Parallel. Low latency for all traffic.**

# Execution: continuous convergence—not a complete spec

The specification becomes precise through the loop—not before it

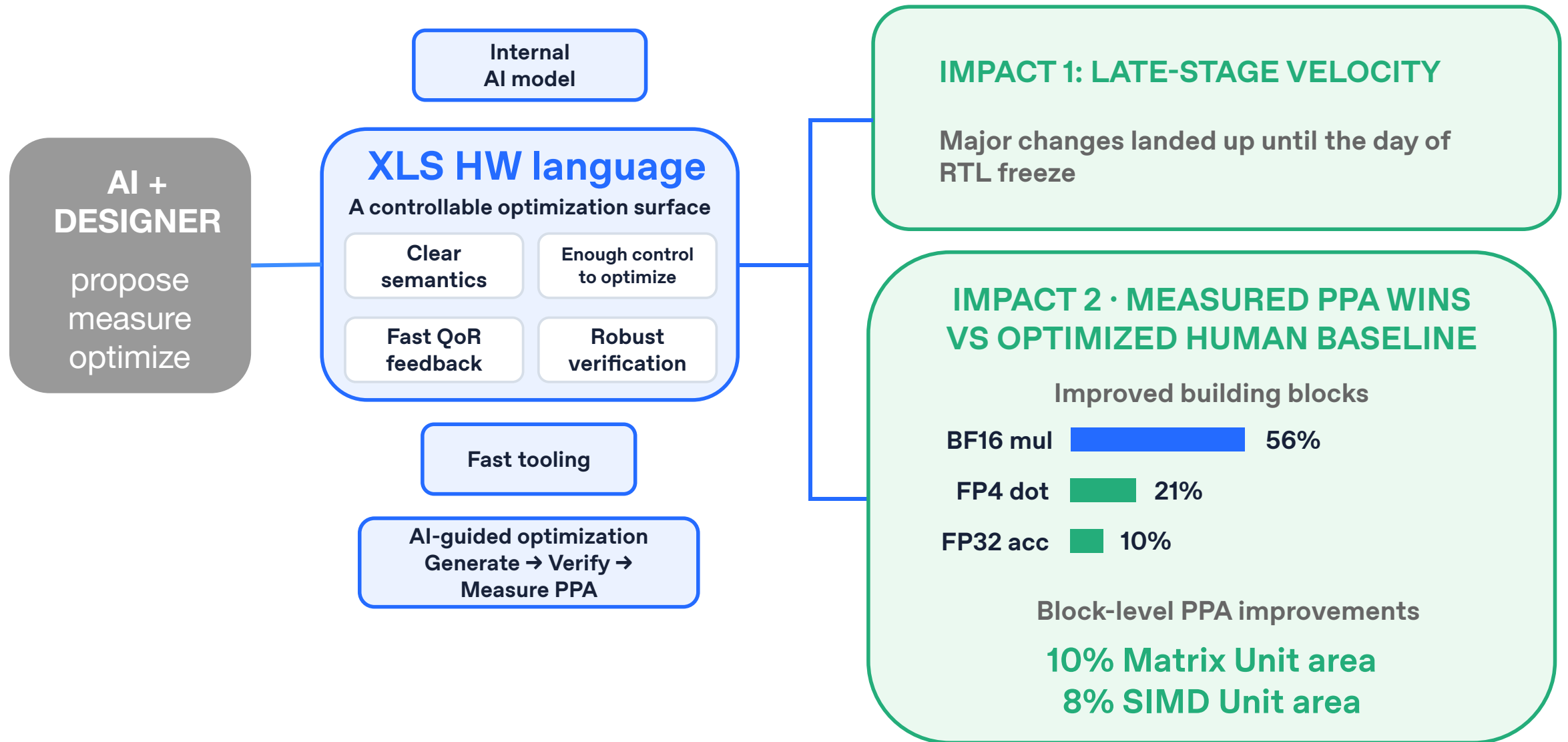


**Win by shortening the loop and running it relentlessly**

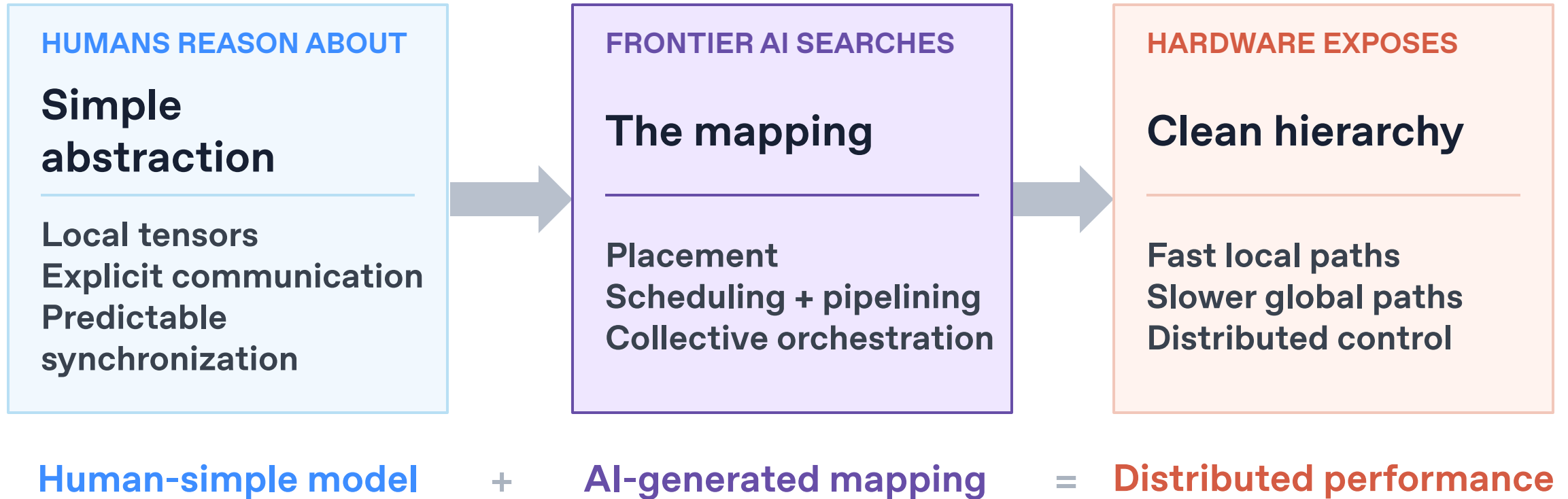
Vision starts the process;  
representative programs, cycle-accurate simulation, QoR, and  
DV/PD/performance feedback converge it.



# AI helped us build Jalapeño faster and made it better



**Architecture model: Simple for humans to understand.  
Easy for frontier AI to program.**

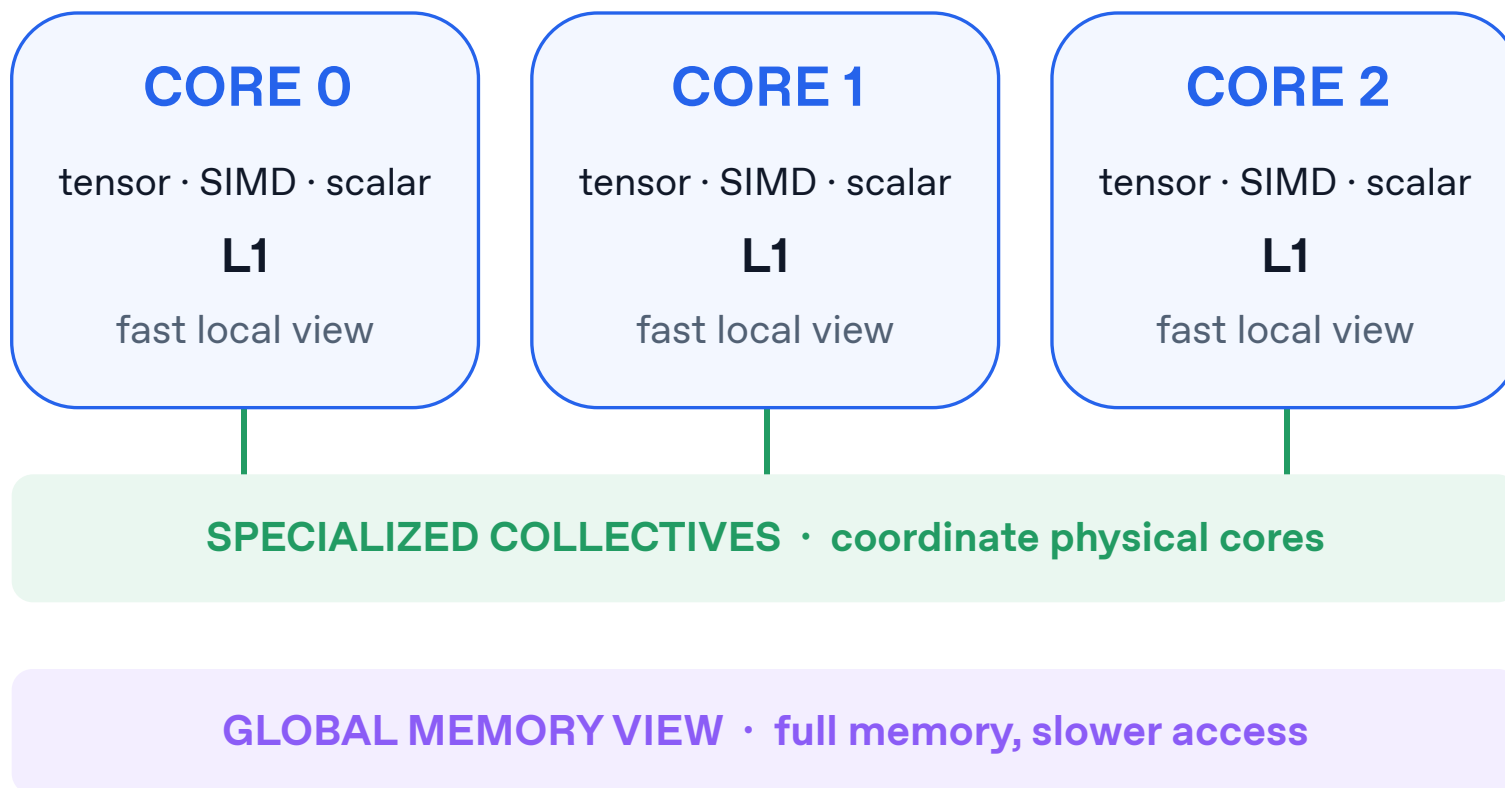


**Spatial programming is onerous for humans. It is easy for frontier AI.**

# Jalapeño is a spatial architecture

Gluon programs each core like a thread block; specialized collectives coordinate across cores.

## GLUON PROGRAMS RUN ON PHYSICAL CORES



## CORE = THREAD BLOCK

Multiple engines share L1 and a fast local memory view.

## SPECIALIZED COLLECTIVES

Coordinate work and data across physical cores.

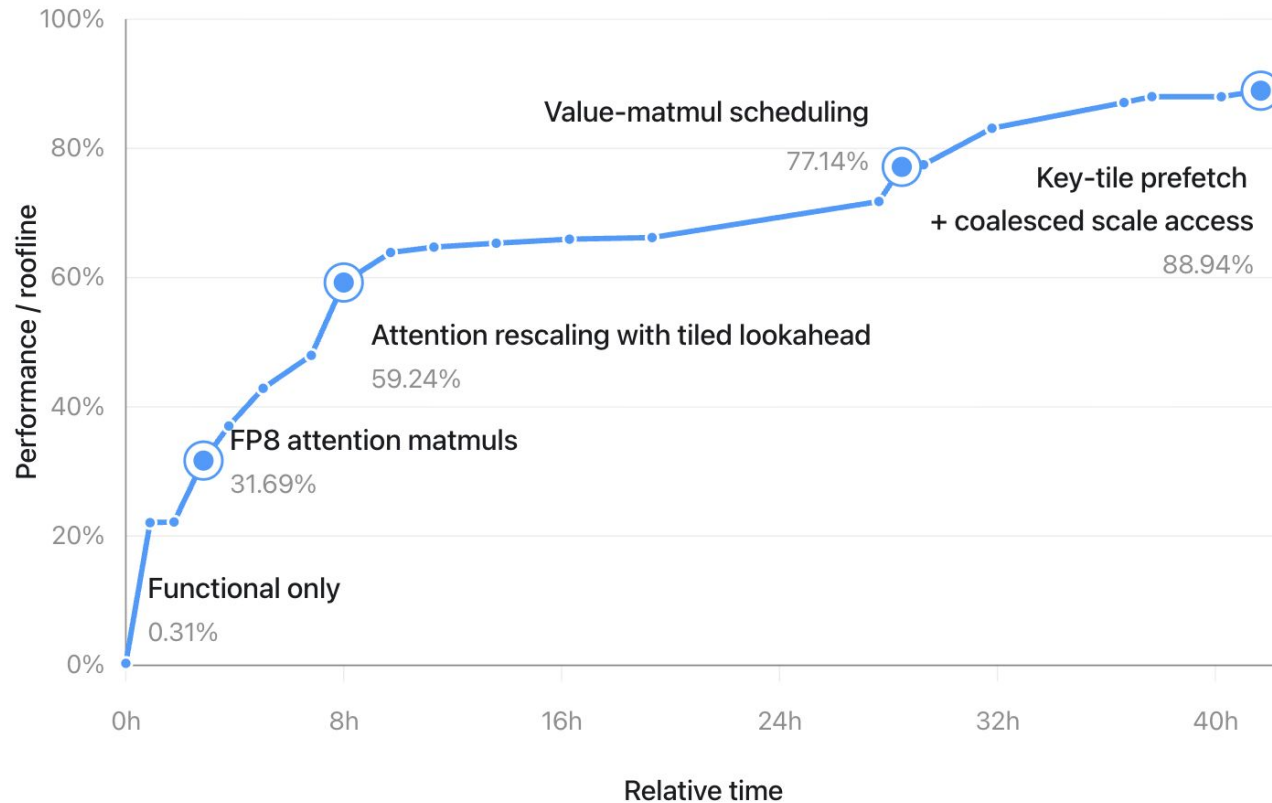
## TENSORINFO

Captures tensor layouts and physical placement explicitly.

**Compute locally. Use global memory only when needed.**

# AI turns functional kernels into high-performance implementations

DeepSeek MLA performance improvement



Functional kernel /  
executable tests

Internal  
harness/model  
with chip/simulator

Optimized kernel  
validated e2e on chip

DeepSeek kernels

AI drove most of the optimization from a functional baseline.

GPT-OSS blocks

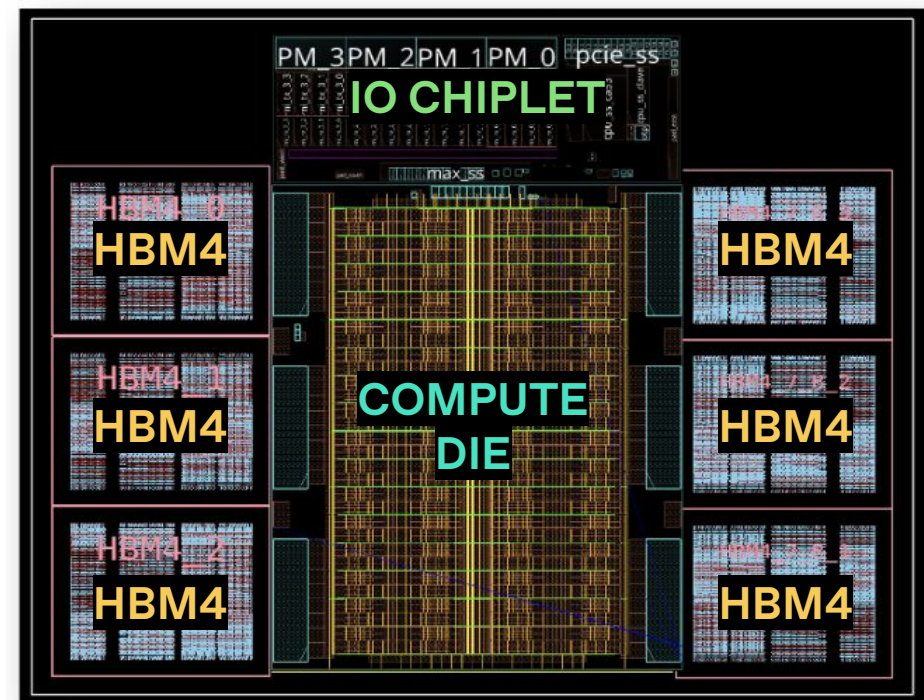
**1.5-1.8× faster** than the existing expert-written implementation on attention and MoE blocks.

## Autonomous optimization. Beyond expert performance.

Starting from functionally correct implementations, our internal AI system drives performance across individual kernels and complete execution blocks.

# Specifications

| Jalapeño ASIC                |                                                                                                                                                                                   |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Matrix compute               | $\text{mxfp8} \times \text{mxfp8} = 3.4 \text{ PFLOP/s}$<br>$\text{mxfp8} \times \text{mxfp4} = 6.7 \text{ PFLOP/s}$<br>$\text{mxfp4} \times \text{mxfp4} = 13.4 \text{ PFLOP/s}$ |
| Memory system                | 15.4 TB/s, 216 GiB                                                                                                                                                                |
| Scale-up network             | Local = 128 ASICs @ 600 GB/s<br>Global = 2048 ASICs @ 200 GB/s                                                                                                                    |
| Power                        | 700 W                                                                                                                                                                             |
| Jalapeño system (2048 ASICs) |                                                                                                                                                                                   |
| Matrix compute               | $\text{mxfp4} \times \text{mxfp4} = 27 \text{ EFLOP/s}$                                                                                                                           |
| Memory system                | 32 PB/s, 432 TiB                                                                                                                                                                  |



Package floorplan

**Most important spec: perf/W on end-to-end workloads**

# You Can Just Build Things ... Chips

OpenAI



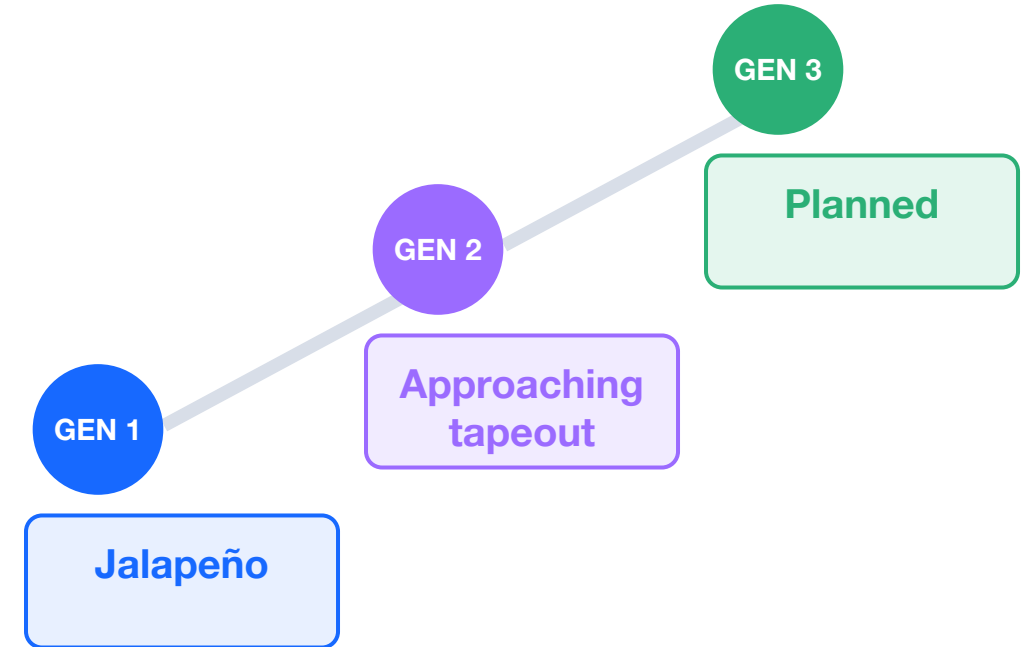
# You Can Just Build Things ... Chips

## WHAT JALAPEÑO UNLOCKS

- 01 ECONOMICAL LOW-LATENCY SERVING**  
Frontier models become practical at interactive SLA latency.
- 02 BETTER PERFORMANCE PER WATT**  
The advantage extends to non-interactive inference as well.
- 03 ARCHITECTURE UNLOCKS HBM**  
The main bottleneck is exposing aggregate HBM bandwidth—not adding more raw bandwidth.

## A MULTI-GENERATION ROADMAP

Each generation compounds our learnings



Jalapeño is our first take. Gen 2 already improves on it, and the path to Gen 3 is taking shape.

**Thanks to the entire OpenAI team + our incredible partners, especially  
Broadcom & Celestica**

# Appendix. Jalapeño STP vs. GB300 MTP using all-in utility MW

## Output token throughput per all-in utility MW vs. interactivity

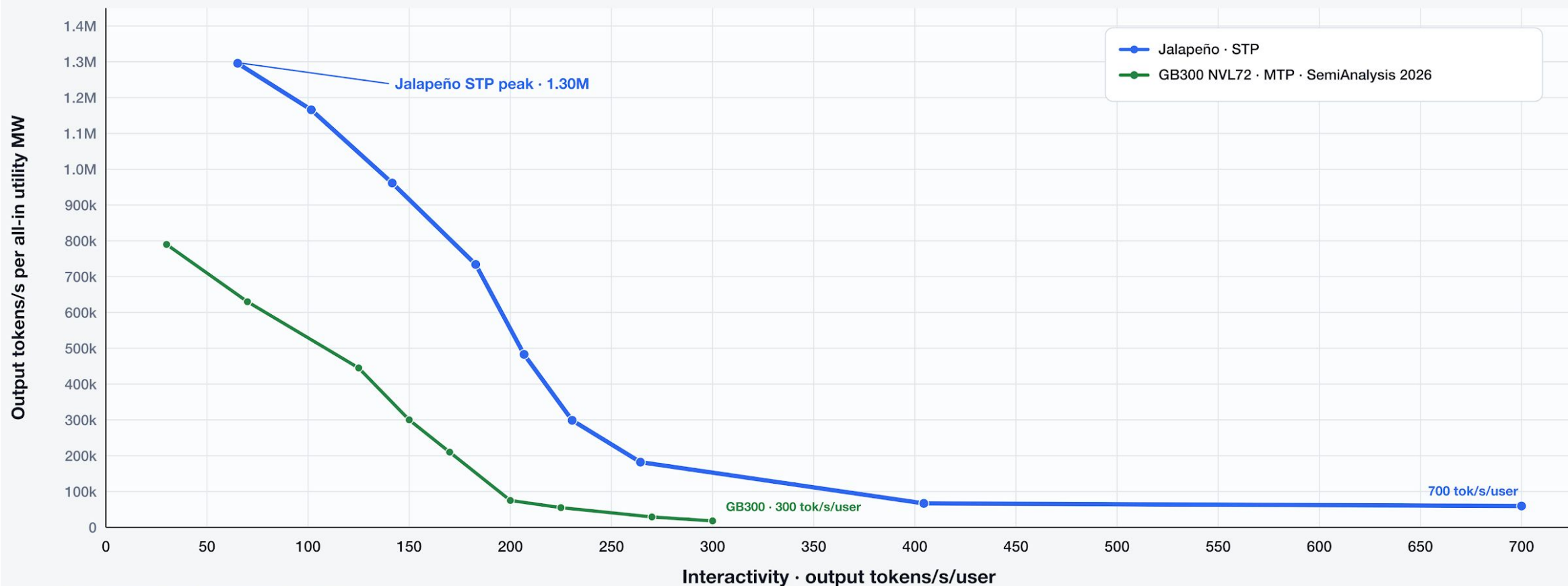
DeepSeek R1 0528 · 671B · FP4 · 8K input / 1K output · GB300 benchmark

All-in utility power / accelerator:

Jalapeño STP 1.18 kW

GB300 MTP 2.553 kW

Jalapeño STP vs. GB300 MTP · output tokens only · all-in utility power



GB300 benchmark: SemiAnalysis InferenceX, July 2026.



# Appendix. Jalapeño STP vs. GB300 MTP at matched speed

## OUTPUT THROUGHPUT AT MATCHED INTERACTIVITY

DeepSeek R1 0528 · output tokens/s per all-in utility MW

Jalapeño STP 1.18 kW | GB300 MTP 2.553 kW



GB300 benchmark: SemiAnalysis InferenceX, July 2026.